Desarrollo de un escáner ligero de seguridad web orientado a PYMEs peruanas

Subtítulo: Prototipo modular en Python y evaluación experimental controlada

Universidad: Universidad Nacional Mayor de San Marcos — Universidad del Perú, Decana de América

Facultad: Facultad de Ingeniería de Sistemas e Informática

Escuela: Escuela Profesional de Ingeniería de Sistemas

Tesista: Emerson Palacios Calle

Asesor: Carlos Edmundo Navarro Depaz

Dedicatoria

Espacio reservado para el texto del tesista.

Agradecimientos

Espacio reservado para el texto del tesista.

Resumen

La presencia web de una parte de las PYMEs peruanas motiva examinar una posible herramienta de revisión inicial, pero no se ha medido aquí la exposición de esas empresas ni sus condiciones de adopción. El objetivo fue desarrollar y evaluar técnicamente un escáner modular en Python que informe indicios seleccionados de riesgo web con un alcance y costo de ejecución declarados. Se integraron reconocimiento de WordPress, Joomla y Laravel, comprobaciones GET de reflejo HTML y errores SQL, observación de cabeceras y reportes de texto y JSON. La evaluación vinculó requisitos, código, pruebas automatizadas y dos bancos locales con verdad de referencia derivada de sus fuentes.

En el banco Docker revisado, 45 procesos sobre nueve casos produjeron 15 decisiones positivas y 15 negativas correctas para cinco señales; precisión y recall fueron 1,00 solo en ese conjunto. Los tres CMS compartían los mismos endpoints PHP de seguridad. Un segundo banco de siete mecanismos independientes, con 35 procesos, obtuvo 2 VP, 1 FP, 10 VN y 1 FN frente a verdad de código; precisión y recall fueron 0,667. El falso positivo surgió de un mensaje SQL simulado y el falso negativo de una consulta vulnerable con error oculto. La mediana de tiempo de pared del segundo banco fue 0,4307 s, frente a 0,0269 s de tiempo interno; el pico de memoria mediano fue 30,21 MiB en la máquina local. El aporte es un artefacto acotado y un protocolo que conserva datos crudos y revela errores de sus reglas. «Ligero» significa una URL y hasta diez parámetros GET por proceso, con respuesta limitada a 1 MB; «bajo costo» es una intención de diseño, sin medición del costo total; «orientado a PYMEs» identifica la población prevista. No se comparó con otra herramienta ni se validó con PYMEs.

Palabras clave: seguridad web; OWASP; escáner modular; XSS reflejado; inyección SQL; evaluación reproducible.

Abstract

The web presence of some Peruvian small and medium-sized enterprises motivates examining a possible tool for initial security review, but this study did not measure those firms' vulnerability exposure or adoption conditions. The objective was to develop and technically evaluate a modular Python scanner that reports selected web risk indications with a declared scope and execution cost. The artifact integrates recognition of WordPress, Joomla, and Laravel, GET checks for HTML reflection and SQL errors, security header observations, and text and JSON reports. Evaluation linked requirements, code, automated tests, and two local test sets whose ground truth derives from their source code.

In the revised Docker set, 45 processes over nine cases yielded 15 correct positive and 15 correct negative decisions for five signals; precision and recall were 1.00 only in that set. The three CMS shared the same PHP security endpoints. A second set of seven distinct mechanisms, with 35 processes, yielded 2 true positives, 1 false positive, 10 true negatives, and 1 false negative against code-defined ground truth; precision and recall were 0.667. The false positive arose from a simulated SQL error message and the false negative from a vulnerable query with a hidden error. Median wall time in the second set was 0.4307 s versus 0.0269 s of internal time; median peak process memory was 30.21 MiB on the local machine. The contribution is a bounded artifact and a protocol that preserves raw data and exposes rule failures. “Lightweight” means one URL and up to ten GET parameters per process, with a 1 MB response limit. “Low cost” is a design aim without a total cost measurement, and “SME oriented” identifies the intended population. No reference tool comparison or SME validation was performed.

Keywords: web security; OWASP; modular scanner; reflected XSS; SQL injection; reproducible evaluation.

Índice de tablas

• Tabla 2.1. Términos y definiciones operativas — p. 10.

• Tabla 3.1. Condiciones de comparación experimental — p. 13.

• Tabla 4.1. Decisiones de diseño y evidencia — p. 15.

• Tabla 4.2. Campos del hallazgo de seguridad — p. 16.

• Tabla 4.3. Trazabilidad de objetivos — p. 17.

• Tabla 5.1. Estructuras en memoria — p. 19.

• Tabla 5.2. Requerimientos funcionales — p. 20.

• Tabla 5.3. Requerimientos no funcionales — p. 20.

• Tabla 5.4. Módulos y pruebas — p. 23.

• Tabla 6.1. Señales en el banco Docker — p. 24.

• Tabla 6.2. Verdad de código en el banco independiente — p. 25.

• Tabla 6.3. Recursos del banco Docker — p. 25.

• Tabla B.1. Casos Docker y señales esperadas — p. 31.

• Tabla D.1. Corridas Docker completas — p. 32.

• Tabla F.1. Corridas independientes completas — p. 34.

Índice de figuras

• Figura 4.1. Flujo de componentes del escáner — p. 16.

• Figura 5.1. Flujo de ejecución por URL y módulos — p. 19.

Índice general

Capítulo 1. Planteamiento del problema....5
Capítulo 2. Marco teórico....9
Capítulo 3. Estado del arte y método de investigación....12
Capítulo 4. Aporte teórico....15
Capítulo 5. Aporte práctico....18
Capítulo 6. Análisis de resultados....24
Conclusiones....27
Recomendaciones....28
Trabajos futuros....29
Referencias bibliográficas....30
Anexos....31

Capítulo 1: Planteamiento del problema

1.1. Antecedentes

La presencia digital de una parte de las empresas pequeñas y medianas peruanas establece una población de interés para la seguridad web. El INEI (2022, p. 42) informó que, entre las empresas privadas de su marco muestral con ventas superiores a 150 UIT, el 24,3 % de las pequeñas y el 43,4 % de las medianas tenían sitio web o presencia en uno en 2019. El indicador no mide vulnerabilidades, costos de seguridad ni uso de WordPress, Joomla o Laravel. Por tanto, aquí se usa para justificar la relevancia de estudiar aplicaciones web en ese segmento, sin convertirlo en una estimación de riesgo.

La OWASP Web Security Testing Guide v4.2 (WSTG-INPV-01 y WSTG-INPV-05) describe procedimientos para identificar entradas reflejadas y examinar consultas SQL mediante respuestas observables. Esas técnicas pueden guiar una revisión inicial, pero una alerta automática exige interpretación del contexto y no equivale por sí misma a una vulnerabilidad explotable. El problema técnico que motiva esta investigación es cómo integrar un subconjunto acotado de tales comprobaciones, reconocimiento tecnológico y observaciones de configuración en una herramienta cuyo alcance y costo de ejecución puedan examinarse.

La motivación de orientar el artefacto a PYMEs peruanas procede del proyecto de investigación: se busca una posible opción de revisión inicial para organizaciones que no operan un proceso especializado de pruebas. Esta necesidad específica, las barreras de adopción y la facilidad de interpretación todavía no fueron medidas con empresas participantes. Tampoco existe una comparación experimental del prototipo con herramientas de referencia. Estas son preguntas de validación, no premisas demostradas.

1.2. Descripción del problema

Una revisión automatizada de una aplicación web debe decidir qué entradas probar, cómo limitar las peticiones, qué observación justifica una alerta y cómo comunicar su incertidumbre. Un escáner de alcance amplio puede producir resultados que no sean directamente comparables con los de una herramienta acotada. En esta tesis el vacío abordable es de diseño y evaluación del artefacto: especificar y comprobar un pipeline modular de una URL por corrida, con señales indicativas y recursos medidos en casos de verdad conocida. La pregunta aplicada a PYMEs permanece abierta: no hay datos de participantes sobre presupuesto, conocimientos, entorno de alojamiento o utilidad del reporte.

1.3. Definición del problema

¿En qué medida un escáner modular de seguridad web en Python puede identificar indicios seleccionados y producir reportes trazables con un costo de ejecución acotado en casos locales de verdad conocida, como etapa previa a evaluar su utilidad para PYMEs peruanas?

1.3.1. Problema principal

- ¿Cómo implementar y evaluar técnicamente un escáner de una URL por corrida que reconozca WordPress, Joomla y Laravel y pruebe señales HTTP seleccionadas en un laboratorio controlado?

1.3.2. Problemas secundarios

- ¿Qué señales de riesgo pueden probarse mediante GET de forma acotada y sin modificar datos?
- ¿Cómo se organizan los módulos y las salidas para mantener trazabilidad entre observación y recomendación?
- ¿Qué resultados de detección, tiempo y memoria se obtienen en casos positivos y negativos conocidos?
- ¿Qué inferencias permiten los casos locales y qué evidencia adicional se necesita para comparar herramientas y evaluar la utilidad en PYMEs?

1.4. Objetivos

1.4.1. Objetivo general

Desarrollar y evaluar técnicamente un escáner web modular en Python para indicios seleccionados de XSS reflejado, SQLi basada en errores y configuración de cabeceras, con reconocimiento de WordPress, Joomla y Laravel, estableciendo el alcance de sus resultados y las pruebas adicionales necesarias para valorar su uso en PYMEs peruanas.

1.4.2. Objetivos específicos

1. Seleccionar y justificar un subconjunto de pruebas según la WSTG, la observabilidad HTTP, el riesgo operativo y el costo medible de ejecución.
2. Diseñar una arquitectura modular con módulos seleccionables, CLI y salidas de texto y JSON.
3. Implementar reconocimiento y perfilado de WordPress, Joomla y Laravel; XSS reflejado indicativo, SQLi no destructiva basada en errores y análisis de cabeceras.

4. Evaluar el prototipo con casos locales positivos y negativos de verdad conocida, y registrar por separado precisión, recall, tiempo y pico de memoria junto con sus límites de generalización.
5. Documentar el uso y producir reportes con evidencia, recomendación y límites explícitos; definir un protocolo posterior de comparación con herramientas y de evaluación con usuarios de PYMEs.

1.5. Justificación

El aporte técnico consiste en una arquitectura modular de pruebas seleccionables, un contrato común de hallazgos que separa observación de interpretación, y una evaluación reproducible que conserva etiquetas, comandos y salidas crudas. La elección de pruebas se basa en técnicas documentadas por la WSTG y en el límite de peticiones GET; no se afirma que cubra las vulnerabilidades más frecuentes de PYMEs peruanas. El aporte metodológico es hacer explícita la unidad de análisis y las condiciones bajo las cuales una cifra de detección puede sostenerse.

«Ligero» significa aquí: una URL inicial por proceso, como máximo diez parámetros GET y un cuerpo de respuesta de hasta 1 MB, con módulos seleccionables y medición de tiempo y memoria. No significa capacidad demostrada para rastrear un sitio completo ni para ejecutarse dentro de hosting compartido. «Bajo costo» es una meta de diseño: instalación, mantenimiento, revisión humana y operación no se cuantificaron, y la distribución del prototipo aún no tiene condiciones de licencia definidas. «Orientado a PYMEs» designa una población y un caso de uso propuestos, no una propiedad validada. Esta delimitación permite someter el diseño a revisión del tutor antes de una comparación o estudio con empresas.

1.6. Alcances y limitaciones

Alcances:

- Pruebas GET de solo lectura de un subconjunto relacionado con OWASP Top 10 y la WSTG.
- Tecnologías objetivo: WordPress, Joomla, Laravel.
- Entornos de prueba en Docker Compose replicables.
- Métricas cuantitativas: precisión, recall, tiempo, carga.

Limitaciones:

- No incluye pruebas destructivas (no se ejecutan payloads que modifiquen datos).
- Cobertura limitada a vectores comunes y de bajo riesgo operativo.
- La herramienta no reemplaza auditorías profesionales.

- La evaluación técnica inicial usa entornos locales controlados; una eventual validación con PYMEs requiere participantes, consentimiento y protocolo documentados.

1.7. Propuesta de tesis

Título propuesto para la evidencia disponible: Desarrollo de un escáner ligero de seguridad web orientado a PYMEs peruanas: prototipo modular en Python y evaluación experimental controlada. Si se conserva «evaluación comparativa» en el título, se requerirá ejecutar una herramienta de referencia bajo el mismo protocolo y entorno antes de sostener esa promesa. Ambas alternativas se presentan al tutor en el informe de revisión.

1.8. Organización de la tesis

El capítulo 2 presenta los conceptos y fuentes que sustentan el alcance; el capítulo 3 expone el estado del arte y el método; el capítulo 4 define la arquitectura propuesta; el capítulo 5 documenta la implementación y el protocolo de evaluación; el capítulo 6 presenta únicamente resultados de ejecuciones reproducibles. Las conclusiones responden a los objetivos dentro de esos límites; los anexos conservan la configuración, el instrumento y las corridas completas.

Capítulo 2: Marco teórico

2.1. Revisión de la literatura

La OWASP Web Security Testing Guide (WSTG) v4.2 describe pruebas de entradas y respuestas de aplicaciones web. En WSTG-INPV-01, la identificación de un reflejo es un paso previo al análisis del contexto y del posible impacto de XSS; WSTG-INPV-05 propone examinar cómo una entrada altera consultas SQL y si la aplicación expone errores. La guía informa el diseño de las pruebas, pero no acredita la efectividad de un escáner particular.

Amankwah et al. (2020) compararon ocho escáneres comerciales y abiertos sobre WebGoat y DVWA mediante precisión, recall, índice de Youden y otros criterios. El manuscrito completo difundido por un coautor permite revisar su método. Abualese y Al-Rousan (2023) describen en el resumen editorial seis escáneres y dos aplicaciones vulnerables, con exactitud, precisión y recall como dimensiones. En ambos casos, las aplicaciones, configuraciones y reglas de conteo condicionan el significado de los resultados. Ningún porcentaje de esas investigaciones se traslada al prototipo.

El resumen editorial de Aljebry et al. (2022) describe una comparación de ZAP y Acunetix sobre un sitio de prueba. El de Syarifudin et al. (2025) presenta una evaluación de una institución XYZ mediante OWASP Top 10 2021. Sus poblaciones y propósitos difieren de una evaluación de uso en PYMEs peruanas. Se citan para situar métodos y objetos de estudio; no se les atribuyen algoritmos, costos de operación ni cifras no disponibles en el contenido consultado.

2.2. Desarrollo de los ejes temáticos

2.2.1. Presencia web y población objetivo

El INEI (2022, p. 42) registró en 2019 sitio web o presencia en uno para el 24,3 % de las pequeñas empresas y el 43,4 % de las medianas dentro del marco de empresas privadas con ventas superiores a 150 UIT. Este indicador establece una población con actividad web observable. No informa prevalencia de fallas, tecnología del sitio, presupuestos ni competencias del personal. Por ello, la tesis orienta el artefacto a un posible uso en PYMEs peruanas sin presentar el laboratorio como muestra de esa población.

2.2.2. Categorías y pruebas OWASP

OWASP Top 10 2021 sitúa XSS (CWE-79) y SQLi (CWE-89) en A03:2021 – Injection. Su categoría A05:2021 – Security Misconfiguration sirve de contexto para revisar cabeceras HTTP. El Top 10 clasifica riesgos de alcance amplio; esta tesis implementa solamente reflejo HTML en parámetros GET, errores SQL diferenciales y presencia de cabeceras seleccionadas. El

reconocimiento de WordPress, Joomla y Laravel informa el contexto técnico y no constituye una prueba de vulnerabilidad.

2.2.3. Arquitectura de escáneres

El artefacto separa adquisición de una respuesta inicial, reconocimiento, perfilado, comprobaciones DAST y reportes. Esta separación es una decisión de diseño propia: permite activar módulos, compartir la primera respuesta y asociar cada observación a una regla. La WSTG informa los procedimientos de prueba, mientras que la arquitectura concreta y las firmas se justifican por los requisitos y se contrastan con el código del capítulo 5. La modularidad no implica por sí sola una reducción medida de memoria, tiempo o falsos positivos.

2.2.4. Significado operativo de ligero y bajo costo

La ejecución está acotada a una URL inicial, diez parámetros GET como máximo y respuestas de hasta 1 MB. El proceso reutiliza una sesión HTTP y permite seleccionar módulos. El capítulo 6 mide el tiempo interno y el pico de memoria de corridas locales; el segundo banco registra también el tiempo de pared. La carga para el operador incluye lectura del reporte y confirmación humana. No se pagó una licencia comercial para desarrollar el prototipo; las condiciones de distribución y el costo total de adopción no se definieron ni midieron. Tampoco se ejecutó dentro de un alojamiento compartido ni rastreó un sitio completo.

2.3. Glosario de términos

La tabla 2.1 fija el significado operativo de los términos usados para interpretar las métricas de los capítulos 5 y 6.

Tabla 2.1. Términos y definiciones operativas del estudio

Término	Definición en este estudio
SQL Injection (SQLi)	Alteración de una consulta SQL mediante una entrada incorporada a su sintaxis. La WSTG-INPV-05 distingue la observación de un error de la evaluación de su impacto.
Cross-Site Scripting (XSS)	Inclusión de contenido ejecutable en una respuesta interpretada por el navegador de la víctima. La WSTG-INPV-01 exige analizar el contexto del reflejo.
DAST	Prueba dinámica de una aplicación en ejecución mediante entradas y respuestas observables.
Indicio	Observación compatible con una falla que requiere confirmación contextual o revisión del código.
VP, FP, VN y FN	Clasificaciones frente a una verdad de referencia declarada. Su interpretación depende de si la etiqueta representa una señal o una vulnerabilidad.
Precisión	VP / (VP + FP), definida solo si hay predicciones positivas.

Término	Definición en este estudio
Recall	VP / (VP + FN), definido solo si existen positivos reales.
Exactitud	(VP + VN) / (VP + FP + VN + FN).

Fuente: elaboración propia con base en WSTG v4.2 para XSS y SQLi; fórmulas de clasificación definidas para este estudio.

2.4. Uso de tablas, figuras y citas

Las tablas cuantitativas de esta tesis identifican su unidad de análisis, fuente y regla de cálculo. Las figuras 4.1 y 5.1 representan la arquitectura y el flujo implementados, respectivamente. Los resultados de otros autores se exponen como antecedentes, y los datos del prototipo se presentan únicamente en el capítulo 6 y sus anexos.

2.5. Consistencia lógica

La presencia web documentada por el INEI motiva el caso de uso, pero no establece por sí misma una necesidad de adopción. Las guías OWASP delimitan las técnicas; los requisitos del capítulo 5 fijan qué hace el artefacto; los casos locales comprueban su comportamiento dentro de ese alcance. La comparación con escáneres de referencia y la evaluación con PYMEs requieren protocolos diferentes y no se infieren de la prueba de laboratorio.

2.6. Base conceptual de la investigación

Peffers et al. (2007, pp. 52–56) organizan la investigación de diseño en identificación del problema, objetivos, diseño y desarrollo, demostración, evaluación y comunicación. En esta tesis, el escáner es el artefacto; las corridas y su análisis constituyen demostración y evaluación técnica. Amankwah et al. (2020) ofrecen un antecedente metodológico de comparación de escáneres, mientras que la WSTG v4.2 proporciona procedimientos técnicos. El diseño de firmas, límites de ejecución y reportes es propio y se evalúa por las pruebas descritas, sin atribuirlo a esos autores.

Capítulo 3: Estado del arte y método de investigación

3.1. Selección de técnicas

Se eligieron reconocimiento de señales públicas, dos sondas GET por parámetro para reflejo HTML, dos variaciones de comillas para errores SQL y lectura de cabeceras. WSTG-INPV-01 y WSTG-INPV-05 sustentan la pertinencia de examinar reflejos y construcción de consultas; los marcadores concretos, el umbral de dos respuestas concordantes y el límite de parámetros son decisiones del diseño. La selección prioriza observabilidad sin alterar datos y no se basa en una prevalencia medida de vulnerabilidades en PYMEs.

El escáner evita descubrimiento automático de enlaces y formularios. Esta reducción permite declarar qué superficie fue realmente examinada y establecer un presupuesto de peticiones. También limita el recall frente a entradas ocultas, POST, autenticación y rutas no proporcionadas. El reconocimiento de CMS no condiciona la ejecución de los detectores; sirve para informar al operador sobre señales tecnológicas visibles.

3.2. Herramientas y estudios de referencia

Amankwah et al. (2020) ejecutaron ocho escáneres sobre dos aplicaciones vulnerables; Abualese y Al-Rousan (2023) indican en el resumen editorial una comparación de seis escáneres sobre dos aplicaciones. Ambos antecedentes hacen relevante fijar casos, verdad de referencia y reglas de puntuación. Aljebry et al. (2022) indican en su resumen una comparación entre ZAP y Acunetix. Syarifudin et al. (2025) tratan una evaluación de un sistema institucional, no una comparación generalizable de herramientas. Los resúmenes no permiten importar cifras de eficacia, recursos o usabilidad de esos trabajos al presente prototipo.

OWASP ZAP es la candidata de referencia abierta para una comparación propia, siempre que se fije una versión y se habiliten reglas capaces de probar los mismos vectores. Un escaneo pasivo de cabeceras y un escaneo activo de inyecciones cumplen propósitos distintos; compararlos sin declarar reglas, contexto y presupuesto de peticiones sería equívoco. En el entorno local de esta investigación no se obtuvo una corrida de ZAP bajo el protocolo común. Por ello, «evaluación comparativa» sigue siendo una promesa metodológica pendiente y no un resultado del capítulo 6.

3.3. Protocolo de comparación propuesto

La tabla 3.1 establece las condiciones mínimas para una comparación con una herramienta de referencia. La unidad sería caso por clase de señal y herramienta, con etiquetas revisadas desde el código de cada aplicación, no desde la salida de los escáneres. Se separarían las alertas de configuración de las vulnerabilidades y los indicios de las explotaciones confirmadas.

Tabla 3.1. Condiciones para una comparación experimental con una herramienta de referencia

Dimensión	Condición común	Dato que debe conservarse
Objetivos	Mismas URLs locales positivas y negativas, incluida una entrada difícil por clase	Código, hash, etiqueta y respuesta base
Configuración	Versiones, módulos, reglas activas, autenticación y límites de petición declarados	Comandos, archivos de configuración y reportes crudos
Detección	Mapeo previo de alertas a XSS, SQLi y cabeceras; «sin regla» queda fuera del denominador común	VP, FP, VN y FN por clase y caso
Recursos	Misma máquina, orden de corridas alternado y arranque declarado	Tiempo de pared, tiempo interno disponible, memoria y peticiones
Costo y uso	Licencia oficial a la fecha y tareas con usuarios si se quiere hablar de adopción	Fuente de licencia, participantes, tiempos y respuestas

Fuente: elaboración propia. La tabla describe un protocolo pendiente, no resultados de ZAP ni de otra herramienta.

3.4. Contribución respecto de los antecedentes

El aporte no consiste en descubrir una nueva clase de vulnerabilidad ni en demostrar superioridad frente a escáneres existentes. Integra un conjunto limitado de pruebas en un CLI con reportes que identifican observación, regla y recomendación. Su contribución evaluable es la trazabilidad entre requisito, módulo, verdad de referencia, salida cruda y conclusión. El segundo banco local introduce negativos difíciles para comprobar si las alertas indicativas conservan significado al variar los mecanismos. Esta decisión responde a la dependencia del rendimiento respecto del banco observada en la literatura comparativa.

3.5. Método de desarrollo y evaluación

La investigación se organiza según las seis actividades de diseño descritas por Peffers et al. (2007, pp. 52–56). La identificación del problema y los objetivos aparecen en el capítulo 1; los capítulos 4 y 5 desarrollan el artefacto; el laboratorio del capítulo 6 lo demuestra y evalúa técnicamente; este manuscrito comunica método, resultados y límites. La demostración con casos controlados no sustituye la evaluación de utilidad con PYMEs.

La primera evaluación fijó nueve casos y cinco repeticiones por caso antes del análisis. Tres páginas base sirven al reconocimiento de CMS. En cada uno de tres contenedores se montaron dos endpoints de seguridad, positivo y negativo, que comparten el mismo archivo PHP. La unidad de clasificación es el par caso–código de señal agregado por mayoría; las cinco repeticiones observan estabilidad, no crean cinco casos independientes. Tras detectar el riesgo

de sobreajuste, se ejecutó un segundo banco de siete rutas locales construidas con mecanismos separados: reflejo HTML, texto plano, SQLite con error visible u oculto, consulta parametrizada y mensaje de error simulado. La verdad de este segundo banco se refiere a la vulnerabilidad observable en el código, no a la mera presencia de un mensaje. Ambos bancos y sus denominadores se informan por separado.

3.6. Herramientas de implementación y medición

La ruta evaluada usa Python 3.12, `requests` para HTTP, bibliotecas estándar para URL y expresiones regulares, y una CLI con salida de texto o JSON. Pytest ejercita el código; Docker Compose proporciona WordPress, Joomla, Laravel y MariaDB para el primer banco. El segundo banco usa el servidor HTTP de Python y SQLite en memoria, exclusivamente en `127.0.0.1`. El tiempo interno se mide dentro del proceso del escáner y excluye arranque y serialización; el segundo banco registra además tiempo de pared desde el proceso padre. El pico de memoria es el *Peak Working Set* del proceso Windows. El método y las versiones se conservan en los manifiestos.

3.7. Alcance de la inferencia

Los casos locales prueban que el prototipo ejecuta sus reglas y exponen dónde estas confunden una señal con una vulnerabilidad o no detectan una entrada vulnerable. No proporcionan una tasa poblacional de falsos positivos o negativos, una comparación con ZAP, una medida de facilidad de uso ni una estimación de desempeño en alojamiento compartido. Las conclusiones se restringen a esos bancos y motivan la comparación y el estudio con PYMEs propuestos en el anexo E.

Capítulo 4: Aporte teórico

Este capítulo formaliza las decisiones de arquitectura que conectan los objetivos con los componentes del prototipo. El capítulo 5 contrasta ese diseño con la implementación y el capítulo 6 con la evidencia experimental.

4.1. Definición de la solución, de la arquitectura, de la metodología

La solución implementada es un escáner web organizado en cinco etapas seleccionables desde el CLI. Esta organización es una decisión de diseño propia; no se atribuye a los artículos cuyos textos completos no se han examinado:

1. **Reconocimiento (`recon`):** infiere la tecnología base a partir de una respuesta HTTP. No descubre endpoints ni la superficie completa de ataque.
2. **Perfilado de CMS (`cms`):** extrae plugins y tema visibles de WordPress, componentes y plantillas visibles de Joomla, y consigna cuando Laravel no expone versión.
3. **Módulos DAST selectivos (`xss`, `sqli`):** ejecutan pruebas indicativas de XSS reflejado y SQLi basada en errores con GET; sus algoritmos se explican en 5.2.2.
4. **Verificación de configuración (`headers`):** revisa cabeceras de seguridad HTTP (CSP, HSTS, X-Frame-Options, etc.).
5. **Motor de reportes:** consolida hallazgos en texto o JSON. La comprensión por administradores todavía no se ha evaluado.

Los módulos DAST contrastan la respuesta base con dos variantes de cada parámetro antes de reportar. El laboratorio confirmó su operación en los casos construidos, pero no comparó una versión de una sola fase; por tanto, no se afirma que las dos variantes reduzcan falsos positivos.

La secuencia de diseño y evaluación sigue las actividades de Peffers et al. (2007) dentro del alcance técnico definido en 3.5. Los manifiestos y las salidas crudas permiten revisar las decisiones de evaluación sin depender solo del reporte final.

4.2. Fundamentación de la solución, de la arquitectura, de la metodología

Cada decisión de diseño de 4.1 se justifica por un requisito o una hipótesis evaluable. La tabla 4.1 distingue la función implementada del beneficio que aún exigiría otra medición.

Tabla 4.1. Decisiones de diseño y evidencia disponible

Decisión de diseño	Fundamento	Referencia
Módulos activables	Ejecutar solo pruebas elegidas y limitar el trabajo	RF-07; selección verificada en CLI; ahorro relativo sin medir
Comparación de respuestas para SQLi	Buscar señales sin alterar datos del objetivo	RF-03 y RNF-04; positivo y negativo Docker evaluados

Decisión de diseño	Fundamento	Referencia
Dos variantes de prueba	Requerir observaciones concordantes para reportar	RF-05; funcionamiento observado; efecto causal sobre FP sin medir
Ejecución selectiva	Ofrecer control de alcance	RNF-01 y RNF-02; costo relativo entre subconjuntos sin medir
Pruebas del alcance definido	Cubrir XSS reflejado, SQLi, reconocimiento, CMS y cabeceras	Objetivos 1 y 3; 45 procesos Docker y 35 procesos del banco adicional
Reportes con evidencia	Comunicar observación, límite y recomendación	Objetivo 5; comprensión por usuarios no evaluada

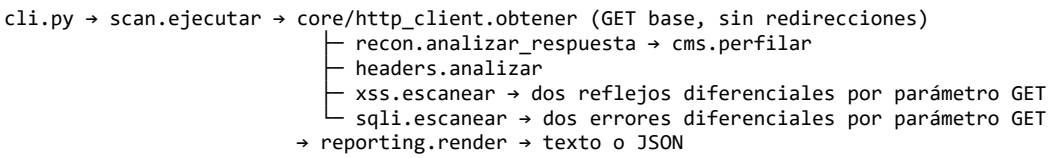
Fuente: elaboración propia a partir de requisitos de 5.1.4, código y corridas de 6.1.

La tabla enlaza cada componente con un requisito y la evidencia correspondiente. Las barreras económicas y de comprensión del usuario todavía no tienen medición directa.

4.3. Definición de la arquitectura de datos, sistemas y tecnológica a usarse

La figura 4.1 muestra el orden de componentes y la reutilización de la respuesta inicial.

Figura 4.1. Flujo de componentes del escáner implementado.



Fuente: elaboración propia a partir de `src/scanner/scan.py` y los módulos invocados.

La segunda observación se implementa dentro de cada detector porque XSS compara marcadores HTML y SQLi compara familias de error. Un componente genérico para ambas reglas exigiría demostrar que preserva esa distinción.

Arquitectura de datos implementada: `HallazgoRecon` contiene tecnología, versión, confianza, evidencia, método y detalles. Los módulos de seguridad producen `HallazgoSeguridad`; `ResultadoEscaneo` reúne estos objetos con URL, estado HTTP, duración, módulos y cantidad de peticiones. La tabla 4.2 enumera los campos de cada hallazgo de seguridad.

Tabla 4.2. Campos del hallazgo de seguridad

Campo	Descripción
`codigo`, `titulo`	Identificador estable y nombre legible de la observación
`modulo`	Módulo de seguridad que lo generó (`xss`, `sqli` o `headers`)
`severidad`	Clasificación provisional: media para señales DAST y baja para cabeceras
`evidencia`	Resumen de las observaciones diferenciales o de la cabecera ausente; el prototipo aún no guarda request/response crudos
`confianza`	`indicativa` para DAST o `observada` para cabeceras; no implica explotación confirmada

Campo	Descripción
`recomendacion`	Explicación en lenguaje simple + paso de mitigación (objetivo específico 5)
`url`, `parametro`	Objetivo analizado y parámetro GET cuando aplica

Fuente: elaboración propia a partir de `src/scanner/core/findings.py`.

Arquitectura tecnológica observada: Python 3.12 en el entorno local; `requests` para HTTP, `argparse` para CLI y biblioteca estándar para URL, patrones y JSON. `aiohttp`, Beautiful Soup, lxml y colorama están declaradas como dependencias, pero esta ruta de escaneo no las usa. Docker Compose ejecutó WordPress, Joomla, Laravel y MariaDB para la evaluación local. No hay documentación Sphinx/MkDocs generada.

La suite local se ejecutó con Python 3.12. La compatibilidad con otras versiones y el despliegue en hosting compartido no se comprobaron.

4.4. Mapeo objetivo específico → componente de arquitectura → capítulo

La tabla 4.3 permite seguir cada objetivo específico hasta su componente y su evaluación.

Tabla 4.3. Trazabilidad de objetivos, arquitectura y evidencia

Objetivo específico (1.4.2)	Componente de arquitectura (4.1/4.3)	Capítulo que lo sustenta
1. Seleccionar y justificar pruebas por alcance, observabilidad y riesgo operativo	Selección en 2.2.2 y protocolo en 5.2.1	Caps. 2, 3 y 5; costo de ejecución local en cap. 6
2. Diseñar arquitectura modular, CLI y salidas	Pipeline de cinco etapas y reportes	Caps. 4 y 5; funcionamiento en cap. 6
3. Implementar XSS, SQLi, reconocimiento, cabeceras y CMS	`modules/{recon,cms,xss,sqli,headers}`	Cap. 5; pruebas en `tests/`
4. Evaluar en entornos locales con métricas	Banco Docker y banco de mecanismos independientes; instrumentación de tiempo y memoria	Caps. 5 y 6; datos en anexos
5. Documentar uso, reportar evidencia y definir validación posterior	Manual, motor de reportes y protocolos propuestos	Cap. 5, anexos A y E; comprensión humana pendiente

Fuente: elaboración propia a partir de objetivos 1.4.2, arquitectura 4.1 y resultados del capítulo 6.

Capítulo 5: Aporte práctico

El prototipo implementa el alcance técnico especificado y se evaluó mediante los dos bancos locales descritos en 3.5. Esta sección expone su diseño, modelo de datos, requisitos, algoritmos y pruebas.

5.1. Diseño de la solución

5.1.1. Desarrollo de diagramas de casos de uso

Caso de uso 1 — Ejecutar un escaneo:

- **Actor previsto:** administrador de una PYME con permiso sobre el sitio; su interacción no se ha observado con participantes.
- **Flujo principal:** se ejecuta el CLI con la URL autorizada del sitio (`scanner https://mi-sitio.pe`) → el sistema corre los módulos seleccionados → se genera un reporte de texto o JSON.
- **Precondición:** el administrador tiene Python/el binario del escáner instalado y el sitio es accesible desde su red.
- **Postcondición:** existe un reporte con hallazgos (o "sin hallazgos"), cada uno con severidad, evidencia y recomendación.

Caso de uso 2 — Revisar y actuar sobre el reporte:

- **Actor previsto:** administrador de la organización o responsable técnico.
- **Flujo principal:** el administrador abre el reporte generado → lee la explicación en lenguaje simple de cada hallazgo → sigue el paso de mitigación sugerido o lo deriva a un tercero técnico.
- **Postcondición observable:** el reporte presenta evidencia, recomendación y límites de cada señal. La comprensión del administrador requiere una prueba de usuarios posterior.

Caso de uso 3 — Ejecutar un escaneo parcial (solo algunos módulos):

- **Actor previsto:** administrador o responsable técnico.
- **Flujo principal:** se invoca el CLI con `--modules` para ejecutar solo un subconjunto de módulos, por ejemplo `headers`.

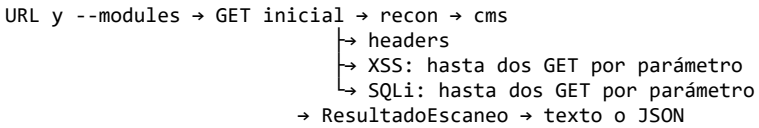
5.1.2. Definición de la arquitectura de la solución, datos y tecnológica

El CLI llama a `scan.ejecutar()`. Este valida la URL y abre una sesión HTTP compartida. `core/http_client.obtener()` solicita por GET la URL inicial, sin seguir redirecciones y con un máximo de 1 MB de cuerpo por respuesta. La misma respuesta alimenta reconocimiento,

perfilado y análisis de cabeceras. Si se activan XSS o SQLi, solo se prueban hasta diez parámetros GET que ya aparecen en la URL introducida; no hay rastreo de enlaces ni envío de formularios.

La figura 5.1 detalla la secuencia de peticiones para una URL y los módulos activos.

Figura 5.1. Flujo de ejecución por URL y módulos.



Fuente: elaboración propia a partir de `src/scanner/scan.py`.

`recon` identifica señales de WordPress, Joomla y Laravel. `cms` extrae plugins y tema visibles de WordPress, componentes y plantillas visibles de Joomla; en Laravel registra que la versión puede no estar expuesta. No infiere versiones ocultas. Los módulos DAST comparan respuestas de la misma URL con valores de prueba distintos y la respuesta inicial. La integración se comprobó con 20 pruebas automatizadas, 45 corridas Docker y 35 corridas del banco adicional.

La herramienta opera en memoria y solo crea un archivo si se usa `--output`. La serialización JSON conserva el resultado estructurado. Los experimentos guardan por separado manifiestos y salidas de cada proceso; el escáner no persiste cuerpos HTTP completos.

5.1.3. Modelo de datos sin base de datos propia

El escáner no necesita una base de datos persistente: cada ejecución procesa una URL, mantiene objetos Python en memoria y opcionalmente escribe un reporte de texto o JSON. Las bases MariaDB de los contenedores WordPress, Joomla y Laravel pertenecen a los objetivos de prueba, no al escáner. Este modelo reduce dependencias de despliegue, pero impide consultar históricos hasta que se conserve una colección externa de reportes. La tabla 5.1 resume las estructuras del proceso.

Tabla 5.1. Estructuras en memoria y uso

Estructura	Campos principales	Uso
`RespuestaHTTP`	URL, estado, cabeceras, cookies y texto acotado a 1 MB	Entrada compartida del análisis; no se persiste
`HallazgoRecon`	tecnología, versión opcional, confianza de firma, evidencia, método y detalles	Salida de `recon` y `cms`; `None` si no hay coincidencia
`HallazgoSeguridad`	código, módulo, severidad, título, evidencia resumida, recomendación, URL, parámetro opcional y confianza	Observación de XSS, SQLi o cabeceras; indicativa salvo ausencia de cabecera observada
`ResultadoEscaneo`	objetivo, HTTP, reconocimiento, lista de hallazgos, duración, peticiones y módulos	Contrato de entrada del reporte de texto o JSON

Fuente: elaboración propia a partir de `src/scanner/core/` y `src/scanner/scan.py`.

No se almacena una tabla de usuarios, sesiones o vulnerabilidades en una base de datos propia. El JSON opcional representa una ejecución, no una base de datos relacional. La unidad de evaluación futura debe identificar cada caso por objetivo, parámetro y tipo de prueba para calcular métricas sin duplicados.

5.1.4. Determinación de los requerimientos funcionales y no funcionales

La tabla 5.2 enumera los requerimientos funcionales implementados.

Tabla 5.2. Requerimientos funcionales del prototipo

ID	Requerimiento
RF-01	El sistema debe aceptar una URL objetivo por línea de comandos y ejecutar el pipeline de módulos seleccionados.
RF-02	El sistema debe identificar el CMS (WordPress, Joomla o Laravel) y su versión cuando sea detectable.
RF-03	El sistema debe reportar indicios de XSS reflejado y SQLi basada en errores mediante pruebas GET de solo lectura, sin atribuir explotación confirmada.
RF-04	El sistema debe verificar la presencia/ausencia de cabeceras de seguridad HTTP estándar.
RF-05	El sistema debe aplicar una validación en dos fases antes de reportar cualquier hallazgo de los módulos DAST.
RF-06	El sistema debe generar un reporte legible que incluya, por cada hallazgo: severidad, evidencia y recomendación de mitigación en lenguaje simple.
RF-07	El sistema debe permitir activar/desactivar módulos individualmente vía CLI.

Fuente: especificación propia contrastada con `src/scanner/` y pruebas automatizadas.

La tabla 5.3 separa las metas no funcionales de sus mediciones disponibles.

Tabla 5.3. Requerimientos no funcionales y criterios de medición

ID	Requerimiento	Umbral
RNF-01	Consumo de memoria	<500 MB RAM durante un escaneo completo
RNF-02	Tiempo de escaneo	Medir duración interna y, cuando se disponga, tiempo de pared por URL con parámetros y módulos declarados
RNF-03	Comprensión del reporte	Registrar éxito en tareas de interpretación y tiempo empleado mediante estudio posterior con usuarios; sin umbral validado aún

ID	Requerimiento	Umbral
RNF-04	No destructividad	Cero payloads que modifiquen, borren o corrompan datos del objetivo (sección 1.6)
RNF-05	Cobertura de pruebas automatizadas	>80% de cobertura de código (Pytest)
RNF-06	Portabilidad	Compatibilidad prevista en Python 3.9+; por ahora solo se verificó Python 3.12 en Windows

Fuente: metas de diseño propias; ver resultados medidos y límites en 6.2 y 6.3.

RNF-01 se cumplió en las corridas locales de una URL (pico máximo de 30,76 MiB en el banco Docker revisado), sin inferir el mismo resultado en hosting compartido. RNF-05 se verificó con 92 % de cobertura total y 20 pruebas aprobadas. RNF-02 dispone de tiempo interno en ambos bancos y de tiempo de pared en el banco adicional. RNF-03 y RNF-06 requieren usuarios y sistemas distintos, respectivamente.

5.2. Implementación de la solución

5.2.1. Definición de métricas y pruebas del sistema

El primer banco usa tres instalaciones Docker locales: WordPress, Joomla y Laravel con MariaDB. En cada contenedor se montaron dos páginas PHP controladas: la positiva refleja `q` sin codificar, concatena `q` en una consulta SQL de solo lectura y omite tres cabeceras; la negativa codifica HTML, usa una consulta parametrizada y envía esas cabeceras. La verdad de referencia se deriva de `docker/lab/scanner_case.php`, cuyo hash figura en el manifiesto de `tests/results/20261013T203602Z`. La página principal de cada CMS se evalúa por separado para reconocimiento. Los endpoints PHP son instrumentación común a los tres entornos; no representan fallas de los CMS instalados ni de PYMEs.

Antes de medir se fijaron cinco corridas por caso y un voto mayoritario por pareja caso–código de señal. Los nueve casos producen 45 procesos CLI; en los seis casos de seguridad se activa el pipeline completo. Las repeticiones observan estabilidad y dispersión, y no se tratan como 45 casos independientes. `runs.jsonl` conserva comandos, salidas JSON y horas; `summary.json` se recalcula con `tests/experiment/analyze_docker.py`. La corrida histórica `20261013T191303Z` y la corrida de la versión revisada `20261013T203602Z` permanecen separadas.

El segundo banco está implementado en `tests/experiment/run_independent.py`. Siete rutas locales usan Python y SQLite en memoria para variar mecanismos positivos y negativos. Cinco repeticiones por ruta producen 35 salidas crudas. La unidad de clasificación es ruta por uno de dos códigos (XSS, SQLi); la verdad de referencia se deriva de las ramas del código. Los casos incluyen una respuesta `text/plain`, un mensaje que imita una excepción SQL sin consulta y una consulta concatenada cuyo error se oculta. `analyze_independent.py` recalcula la tabla 6.2. Los

bancos no se fusionan porque el primero etiqueta señales diseñadas para el detector y el segundo distingue vulnerabilidad de mera señal.

Para cada código se cuentan VP, FP, VN y FN por caso según el voto mayoritario. Precisión = VP/(VP+FP), recall = VP/(VP+FN), exactitud = (VP+VN)/(VP+VN+FP+FN). Si un denominador es cero, la métrica queda sin definir. La duración interna se mide desde antes de la GET inicial hasta terminar los módulos, sin arranque de Python ni serialización; el segundo banco añade tiempo de pared de cada proceso CLI. El pico de memoria en Windows es `PeakWorkingSetSize`. La medición es por una URL inicial con un parámetro GET, no por un sitio completo ni por una instalación de hosting compartido.

5.2.2. Definición de algoritmos principales

Reconocimiento y perfilado. Una GET inicial produce `RespuestaHTTP`. Se comparan meta generator, cookies y rutas visibles en el HTML con firmas de WordPress, Joomla y Laravel. Se devuelve la primera tecnología coincidente y la versión solo si la señal la expone. `cms.perfilar()` añade detalles visibles sin nueva petición desde el pipeline.

XSS reflejado indicativo. Solo se analiza una respuesta inicial con tipo `text/html` o `application/xhtml+xml`. Para cada parámetro GET existente se generan dos etiquetas aleatorias dentro de elementos SVG con `onload=void(0)`. La primera variante se envía por GET; si el elemento aparece literalmente, se envía la segunda. Se reporta solo si ambos elementos aparecen sin codificar en sus respuestas y ninguno estaba en la respuesta base. El escáner no ejecuta JavaScript ni comprueba el efecto de CSP o del contexto DOM; una coincidencia exige confirmación contextual. HTML escapado y parámetros que no se reflejan quedan fuera de la detección.

SQLi basada en errores indicativa. Se añade una comilla simple y luego una variante con comilla más texto al valor original de cada parámetro GET. Se busca una familia concreta de mensajes de error SQL en ambas respuestas y su ausencia en la base. No se envían sentencias `UPDATE`, `DELETE` ni `DROP`, ni se extraen datos. Un error ocultado por la aplicación o una SQLi ciega puede producir falso negativo; un mensaje generado por otra capa puede producir falso positivo.

Cabeceras. Se normalizan nombres sin distinguir mayúsculas. En HTML se observa CSP y protección de embebido mediante `frame-ancestors` o `X-Frame-Options`; en toda respuesta se observa `X-Content-Type-Options: nosniff`; HSTS solo se comprueba en HTTPS. La ausencia es una recomendación de configuración, no prueba de explotación. Todos los módulos producen observaciones con alcance y limitaciones visibles en el reporte.

La comparación diferencial de XSS y SQLi se implementa dentro de cada módulo, ya que cada uno compara objetos distintos. La arquitectura no presupone que esas sondas conviertan una alerta indicativa en una vulnerabilidad confirmada.

5.2.3. Lista de módulos principales

La tabla 5.4 enlaza módulos, contratos, requisitos y pruebas de la versión evaluada.

Tabla 5.4. Módulos implementados y evidencia de prueba

Módulo	Entrada y salida	Requisito	Estado y prueba
`recon`	`RespuestaHTTP` → `HallazgoRecon` opcional	RF-02	Implementado; fixtures, servidor local y tres páginas base Docker
`cms`	`HallazgoRecon` + HTML → reconocimiento enriquecido	RF-02	Implementado para detalles visibles de WordPress/Joomla y versión no expuesta de Laravel; pruebas unitarias y local
`xss`	URL con GET + respuesta HTML base → lista de `HallazgoSeguridad`	RF-03, RF-05	Implementado para reflejo literal diferencial; texto plano excluido; requiere validación contextual
`sqli`	URL con GET + respuesta base → lista de `HallazgoSeguridad`	RF-03, RF-05	Implementado para errores SQL diferenciales; falso positivo y falso negativo documentados en el banco adicional
`headers`	respuesta base → observaciones de cabeceras ausentes	RF-04	Implementado; positivo y negativo locales; HSTS condicionado a HTTPS
`reporting` y CLI	`ResultadoEscaneo` → texto, JSON o archivo	RF-01, RF-06, RF-07	Implementado; 80 procesos CLI en los dos bancos revisados; comprensión por usuarios pendiente

Fuente: elaboración propia a partir de `src/scanner/`, suite Pytest y manifiestos experimentales.

El escáner limita su superficie a la URL inicial y sus parámetros GET existentes. No rastrea enlaces, formularios, sesiones autenticadas ni aplicaciones de una PYME real. Los resultados de estas pruebas locales de ingeniería no permiten inferir precisión, recall ni usabilidad de campo.

Capítulo 6: Análisis de resultados

6.1. Correspondencia entre predicción y verdad de referencia

La corrida inicial de 45 procesos se conserva en `tests/results/20261013T191303Z`. Su `summary.json` coincidió exactamente con un recálculo desde `runs.jsonl` y `manifest.json`. Después de corregir el detector de XSS para exigir contenido HTML se ejecutó de nuevo el protocolo Docker, sin reetiquetar la corrida inicial: los datos de la versión evaluada aquí están en `tests/results/20261013T203602Z`. El cambio de código queda identificado por los hashes del nuevo manifiesto.

En el banco Docker, cada tecnología aporta una página principal para reconocimiento y dos endpoints de seguridad montados desde el mismo archivo PHP. Las cinco repeticiones de cada uno de los nueve casos fueron concordantes. El reconocimiento acertó en las tres páginas principales, con cinco repeticiones por página. La tabla 6.1 resume las seis páginas de seguridad. La unidad de conteo es caso por código de señal después de votar por mayoría; una cabecera ausente cuenta como observación de configuración y no como vulnerabilidad explotable.

Tabla 6.1. Clasificación de señales en el banco Docker de la versión revisada

Código de señal	VP	FP	VN	FN
Reflejo HTML indicativo de XSS	3	0	3	0
Error SQL indicativo	3	0	3	0
Ausencia de CSP	3	0	3	0
Ausencia de protección contra embebido	3	0	3	0
Ausencia de `X-Content-Type-Options: nosniff`	3	0	3	0
Total de decisiones caso–código	**15**	**0**	**15**	**0**

Fuente: cálculo de `manifest.json` y `runs.jsonl` en `20261013T203602Z` mediante `analyze_docker.py`. La precisión, el recall y la exactitud son 1,00 en este banco diseñado para las reglas del detector. Los tres despliegues no aportan tres implementaciones independientes de XSS o SQLi: reutilizan un positivo y un negativo PHP. Por ello, las 30 decisiones de la tabla no constituyen una muestra de 30 vulnerabilidades distintas ni permiten inferir una tasa de detección en sitios reales.

El banco adicional independiente usa un servidor local de Python y SQLite en memoria, sin los endpoints PHP de los contenedores. Se fijaron siete rutas y dos códigos de indicio; cinco

repeticiones por ruta produjeron 35 procesos. La verdad de referencia se estableció a partir de cada rama del código antes de puntuar la salida. En este banco, «positivo» significa una construcción de entrada que permite modificar la sintaxis SQL o insertar marcado ejecutable en una respuesta HTML sin CSP; «negativo» incluye un mensaje de error simulado sin consulta y un reflejo literal servido como texto plano. No se probó explotación en navegador ni extracción de datos. La tabla 6.2 clasifica 14 decisiones caso–código por mayoría de las cinco repeticiones.

Tabla 6.2. Clasificación frente a verdad de código en el banco independiente

Resultado	VP	FP	VN	FN	Precisión	Recall
Siete rutas × dos códigos	2	1	10	1	0,667	0,667

Fuente: `tests/results/20261013T203541Z-independent/manifest.json` y `runs.jsonl`, recalculados con `analyze_independent.py`. El falso positivo fue `sqli-fake-error`: la respuesta contenía el patrón `sqlite3.OperationalError` tras dos comillas, aunque el servidor no ejecutaba SQL. El falso negativo fue `sqli-hidden`: el código concatenaba la entrada en una consulta de solo lectura, pero ocultaba la excepción. El reflejo literal en `text/plain` no produjo alerta XSS después de exigir contenido HTML. Este banco muestra límites de las reglas, no una tasa estimada para PYMEs o para la web en general. Tampoco debe fusionarse con la tabla 6.1: los bancos tienen implementaciones y etiquetas de verdad diferentes.

6.2. Tiempo y memoria del proceso

La tabla 6.3 presenta recursos del banco Docker revisado. Su duración va desde antes de la primera petición hasta el fin de los módulos; excluye arranque de Python y serialización del reporte. El pico de memoria sí corresponde al proceso completo y se mide como *Peak Working Set* en Windows. Los extremos se informan para mostrar la dispersión sin tratar repeticiones como casos independientes.

Tabla 6.3. Recursos de 45 procesos frescos del banco Docker revisado

Grupo	Corridas	Duración interna mediana; mínimo–máximo	Memoria pico mediana; máximo
Seguridad, pipeline completo de una URL	30	0,0373 s; 0,0167–0,1448 s	30,25 MiB; 30,41 MiB
Reconocimiento y perfilado de página principal	15	0,0830 s; 0,0253–0,1469 s	30,44 MiB; 30,76 MiB

Fuente: `tests/results/20261013T203602Z/summary.json`, recalculable desde las corridas crudas. En el banco independiente, donde se activaron solo XSS y SQLi, la mediana de 35 procesos fue 0,0269 s de tiempo interno y 0,4307 s de tiempo de pared medido por el proceso padre; la memoria pico mediana fue 30,21 MiB. Esas medianas no son una comparación controlada de los módulos entre bancos, porque objetivos y cargas difieren. La diferencia entre tiempo de

pared e interno muestra que omitir arranque y reporte subestima el tiempo que percibe quien invoca el CLI. Ninguna medición incluye latencia de Internet, navegación de un sitio completo, operación en hosting compartido ni tiempo humano de confirmar alertas.

6.3. Pruebas automatizadas y validez del experimento

La suite ejecutada con Python 3.12 y los contenedores activos obtuvo 20 pruebas aprobadas, ninguna omitida y 92 % de cobertura de líneas. Una prueba nueva verifica que el módulo XSS no trate texto plano como HTML. La cobertura representa código ejercitado, no sensibilidad ante fallas reales. Los tres CMS se reconocieron en páginas instaladas localmente; los endpoints PHP no son fallas nativas de WordPress, Joomla o Laravel. El segundo banco amplía mecanismos y negativos difíciles, pero tiene siete rutas construidas por el investigador y dos clases de vulnerabilidad. Su resultado no autoriza intervalos poblacionales ni una afirmación de superioridad frente a otras herramientas.

6.4. Alcance del aporte y evaluación pendiente

El CLI integró una respuesta inicial compartida, módulos seleccionables y reportes de texto y JSON. Las dos sondas por parámetro hicieron observables las reglas de reflejo y error; no se aisló experimentalmente su efecto causal sobre falsos positivos. El mensaje SQL simulado demuestra que concordancia de patrones no acredita una consulta vulnerable, y el error oculto demuestra que ausencia de alerta no implica ausencia de SQLi. La evaluación comparativa con un escáner de referencia, el costo total de adopción y la comprensión del reporte por administradores requieren los protocolos de la tabla 3.1 y del anexo E. Son límites científicos del trabajo presentado al tutor, no resultados implícitos de los ensayos locales.

Conclusiones

1. Para el objetivo de seleccionar pruebas, la WSTG y el alcance de peticiones GET permitieron definir reflejo HTML, error SQL y cabeceras como observaciones reproducibles. Esta selección no se justificó por una prevalencia medida de vulnerabilidades en PYMEs peruanas; su pertinencia para esa población requiere evidencia directa.

2. Para el objetivo de diseño, se implementó una arquitectura con respuesta HTTP inicial compartida, módulos seleccionables, contrato estructurado de hallazgos y dos formatos de reporte. La suite de 20 pruebas y las corridas CLI verifican su integración local. No se aisló experimentalmente el ahorro atribuible a la modularidad.

3. Para el objetivo de implementación, el prototipo reconoció WordPress, Joomla y Laravel en tres páginas principales locales y ejecutó las comprobaciones delimitadas. El detector de XSS se corrigió para no clasificar reflejos de texto plano como HTML. Sus hallazgos XSS y SQLi son indicios que requieren análisis del contexto, no pruebas de explotación.

4. Para el objetivo de evaluación, el banco Docker revisado produjo 15 VP y 15 VN sin errores de clasificación de señales, con precisión y recall de 1,00 en endpoints diseñados para sus reglas. El banco adicional de siete mecanismos obtuvo 2 VP, 1 FP, 10 VN y 1 FN frente a verdad de código, con precisión y recall de 0,667. El FP de SQLi ante un mensaje simulado y el FN ante un error oculto delimitan la capacidad de detección. Las cifras de ambos bancos no son estimaciones de campo ni resultados comparativos con otras herramientas.

5. Para el objetivo de documentación, el CLI genera salidas de texto y JSON con observación y recomendación, y conserva datos crudos para recalcular los resultados. La mediana de tiempo de pared de 0,4307 s en el banco adicional excedió su tiempo interno mediano de 0,0269 s; la medición interna omite costos perceptibles del proceso. Ni la comprensión del reporte, ni el costo total, ni el uso en hosting compartido o en PYMEs se evaluaron. Estos límites definen el siguiente estudio, no una conclusión de utilidad.

Recomendaciones

- Confirmar manualmente las alertas SQL con revisión de consultas o un entorno aislado. El caso con mensaje simulado demostró que dos coincidencias de error pueden ocurrir sin SQLi.
- Incorporar casos de error oculto y pruebas de respuesta diferencial que conserven el carácter de solo lectura; el falso negativo del segundo banco muestra el límite del patrón de error visible.
- Para alegar una ventaja relativa, ejecutar al menos una herramienta de referencia con el protocolo de la tabla 3.1, conservando versiones, reglas, salidas y costos de arranque comparables. No mezclar alertas de distinta categoría en una sola precisión.
- Evaluar el reporte con personas de PYMEs mediante tareas de interpretación y registrar necesidad de apoyo técnico, tiempo y errores. Los resultados de laboratorio no sustituyen esa observación.

Trabajos futuros

1. Añadir análisis contextual de XSS en navegador aislado y un conjunto de respuestas con CSP y ubicaciones HTML diversas, para distinguir reflejo literal de ejecución factible.
2. Investigar SQLi sin error visible con sondas de solo lectura y control de efectos; medir el cambio de recall y falsos positivos frente al banco independiente.
3. Comparar el prototipo con una versión fijada de ZAP bajo el mismo banco, reglas equivalentes y presupuesto de peticiones; informar clases que una herramienta no cubra sin atribuirles acierto o fallo.
4. Aplicar el protocolo de usuarios del anexo E en PYMEs voluntarias y, solo con autorización y consentimiento, examinar aplicaciones propias de los participantes. Separar sus resultados de las corridas locales.

Referencias bibliográficas

- Instituto Nacional de Estadística e Informática. (2022). *Perú: Tecnologías de Información y Comunicación en las Empresas, 2019*. https://www.inei.gob.pe/media/MenuRecursivo/publicaciones_digitales/Est/Lib1906/libro.pdf
- OWASP Foundation. (2021). *OWASP Top 10: The Ten Most Critical Web Application Security Risks*. https://owasp.org/www-project-top-ten/
- OWASP Foundation. (2020). *OWASP Web Security Testing Guide*, versión 4.2. https://wstg.owasp.org/v4.2/
- Peffers, K., Tuunanen, T., Rothenberger, M. A., & Chatterjee, S. (2007). *A design science research methodology for information systems research*. Journal of Management Information Systems, 24(3), 45–77. https://doi.org/10.2753/MIS0742-1222240302
- Abualese, H., & Al-Rousan, T. (2023). *A comparative study of web application security scanners for vulnerability detection*. i-manager's Journal on Software Engineering, 17(4), 1–8. https://doi.org/10.26634/jse.17.4.19813
- Syarifudin, M., Widyawati, L., & Asroni, O. (2025). *Web Security Vulnerability Analysis and Mitigation Based on OWASP TOP 10*. Journal of Artificial Intelligence and Engineering Applications (JAIEA), 4(3), 1829–1834. https://doi.org/10.59934/jaiea.v4i3.1029
- Amankwah, R., Chen, J., Kudjo, P. K., & Towey, D. (2020). *An empirical comparison of commercial and open-source web vulnerability scanners*. Software: Practice and Experience, 50(9), 1842–1857. https://doi.org/10.1002/spe.2870
- Aljebry, A.F., Alqahtani, Y.M., & Sulaiman, N. (2022). *Analyzing Security Testing Tools for Web Applications*. En *International Conference on Innovative Computing and Communications* (Advances in Intelligent Systems and Computing, vol. 1387, pp. 411–419). Springer. https://doi.org/10.1007/978-981-16-2594-7_34

Anexos

Anexo A. Reproducción del prototipo y del laboratorio

El código se instala con `poetry install` y se ejecuta con `poetry run python -m scanner URL --format json`. Para la corrida documentada se usó Python 3.12.0 en Windows. Los tres archivos `docker/<cms>/docker-compose.yml` fijan puertos `127.0.0.1:8081–8083` y montan `docker/lab/scanner_case.php` en dos rutas de solo lectura. Se levantan por separado con `docker compose up -d` en cada carpeta. WordPress se inicializa con `.venv/Scripts/python.exe tests/experiment/install_wordpress.py`; Joomla utiliza variables de instalación automática del contenedor oficial; Laravel se construye con `docker compose up -d --build` y Composer instala Laravel 12 compatible con PHP 8.2. La ejecución no debe dirigirse a sitios de terceros.

El instrumento Docker se ejecuta con `.venv/Scripts/python.exe tests/experiment/run_docker.py --repeticiones 5`. Para recalcular la versión revisada: `.venv/Scripts/python.exe tests/experiment/analyze_docker.py tests/results/20261013T203602Z --output resumen-recalculado.json`. El segundo banco se ejecuta con `.venv/Scripts/python.exe tests/experiment/run_independent.py` y se recalcula con `.venv/Scripts/python.exe tests/experiment/analyze_independent.py tests/results/20261013T203541Z-independent`. Los manifiestos fijan etiquetas, versiones, hashes de código y reglas de agregación; ambos `runs.jsonl` guardan las salidas completas. La suite se ejecuta con `.venv/Scripts/python.exe -m pytest --cov=scanner --cov-report=term-missing -q`. Todos los objetivos son locales.

Anexo B. Instrumento y verdad de referencia

La tabla B.1 identifica las etiquetas fijadas para el primer banco y el código que las origina.

Tabla B.1. Casos Docker y señales esperadas

Caso por CMS	Respuesta a `q`	Consulta SQL	Cabeceras	Etiquetas esperadas
`scanner-vulnerable.php?q=hola`	Salida HTML sin escape	Concatenación de `q` en `SELECT 1`	CSP, protección de embebido y `nosniff` ausentes	`XSS-REFLEJO-INDICATIVO`, `SQLI-ERROR-INDICATIVO`, `HEADER-CSP`, `HEADER-FRAME`, `HEADER-NOSNIFF`
`scanner-safe.php?q=hola`	HTML codificado	Consulta parametrizada	CSP con `frame-ancestors` y `nosniff` presentes	Ninguna de esas cinco señales
Página principal `/`	Instalación estándar local	No se sondea SQL	No entra al conteo de seguridad	WordPress, Joomla o Laravel según contenedor

Fuente: elaboración propia a partir de `docker/lab/scanner_case.php` y manifiesto `20261013T203602Z`.

El mismo archivo PHP se montó en los tres entornos, de modo que la diversidad de casos de seguridad es limitada aunque haya tres CMS. La etiqueta positiva de XSS se refiere al reflejo sin codificar en HTML, no a ejecución demostrada en un navegador. La etiqueta SQLi proviene de una consulta SQL real de solo lectura que produce un error de sintaxis ante comillas, no de una simulación textual del error.

Anexo C. Límites de lectura y trazabilidad

Las carpetas `20261013T191200Z` y `20261013T191209Z` documentan un intento abortado y una corrida piloto Docker. `20261013T191303Z` contiene la evaluación inicial, conservada para auditoría y recalculada sin cambios. Las carpetas independientes `20261013T203317Z`, `20261013T203358Z` y `20261013T203505Z` fueron pilotos excluidos por diseños ambiguos o no exigentes del caso negativo; cada incidencia está documentada en su carpeta. El capítulo 6 usa solo `20261013T203602Z` y `20261013T203541Z-independent`. Las repeticiones de un caso comparten código y objetivo y no se tratan como muestras independientes. El historial de fuentes descartadas y de decisiones experimentales se conserva fuera de la lectura principal, en `docs/notas-verificacion.md`.

Anexo D. Tabla completa de corridas

La tabla D.1 se generó directamente de `tests/results/20261013T203602Z/runs.jsonl`. Las abreviaturas CSP, FRAME y NOSNIFF indican ausencia de esas protecciones; XSS y SQLI son las señales indicativas descritas en 5.2.2. La columna de señales resume la salida sin sustituir el JSON crudo, que conserva evidencia, recomendaciones, tiempo y memoria de cada ejecución.

Tabla D.1. Cuarenta y cinco corridas Docker de la versión revisada

Caso	Rep.	HTTP	Señales / CMS	Tiempo (s)	Pico (MiB)
wordpress-vulnerable	1	200	CSP, FRAME, NOSNIFF, XSS, SQLI	0.0620	30.25
wordpress-vulnerable	2	200	CSP, FRAME, NOSNIFF, XSS, SQLI	0.1448	30.41
wordpress-vulnerable	3	200	CSP, FRAME, NOSNIFF, XSS, SQLI	0.0489	30.21
wordpress-vulnerable	4	200	CSP, FRAME, NOSNIFF, XSS, SQLI	0.0361	30.36
wordpress-vulnerable	5	200	CSP, FRAME, NOSNIFF, XSS, SQLI	0.0286	30.23
wordpress-seguro	1	200	ninguno	0.0191	30.31
wordpress-seguro	2	200	ninguno	0.0425	30.16
wordpress-seguro	3	200	ninguno	0.0183	30.27
wordpress-seguro	4	200	ninguno	0.0237	30.25
wordpress-seguro	5	200	ninguno	0.0208	30.17
wordpress-cms	1	200	WordPress	0.0819	30.61
wordpress-cms	2	200	WordPress	0.1378	30.49
wordpress-cms	3	200	WordPress	0.1469	30.76

Caso	Rep.	HTTP	Señales / CMS	Tiempo (s)	Pico (MiB)
wordpress-cms	4	200	WordPress	0.0707	30.63
wordpress-cms	5	200	WordPress	0.0850	30.61
joomla-vulnerable	1	200	CSP, FRAME, NOSNIFF, XSS, SQLI	0.0797	30.17
joomla-vulnerable	2	200	CSP, FRAME, NOSNIFF, XSS, SQLI	0.0378	30.36
joomla-vulnerable	3	200	CSP, FRAME, NOSNIFF, XSS, SQLI	0.0366	30.34
joomla-vulnerable	4	200	CSP, FRAME, NOSNIFF, XSS, SQLI	0.0308	30.18
joomla-vulnerable	5	200	CSP, FRAME, NOSNIFF, XSS, SQLI	0.0390	30.16
joomla-seguro	1	200	ninguno	0.0450	30.29
joomla-seguro	2	200	ninguno	0.0298	30.28
joomla-seguro	3	200	ninguno	0.0197	30.16
joomla-seguro	4	200	ninguno	0.0167	30.21
joomla-seguro	5	200	ninguno	0.0388	30.29
joomla-cms	1	200	Joomla	0.1432	30.27
joomla-cms	2	200	Joomla	0.0638	30.29
joomla-cms	3	200	Joomla	0.0830	30.36
joomla-cms	4	200	Joomla	0.0965	30.27
joomla-cms	5	200	Joomla	0.0432	30.35
laravel-vulnerable	1	200	CSP, FRAME, NOSNIFF, XSS, SQLI	0.0824	30.16
laravel-vulnerable	2	200	CSP, FRAME, NOSNIFF, XSS, SQLI	0.0271	30.41
laravel-vulnerable	3	200	CSP, FRAME, NOSNIFF, XSS, SQLI	0.0441	30.30
laravel-vulnerable	4	200	CSP, FRAME, NOSNIFF, XSS, SQLI	0.0447	30.16
laravel-vulnerable	5	200	CSP, FRAME, NOSNIFF, XSS, SQLI	0.0441	30.25
laravel-seguro	1	200	ninguno	0.0603	30.21
laravel-seguro	2	200	ninguno	0.0230	30.12
laravel-seguro	3	200	ninguno	0.0368	30.04
laravel-seguro	4	200	ninguno	0.0382	30.34
laravel-seguro	5	200	ninguno	0.0177	30.28
laravel-cms	1	200	Laravel	0.1235	30.50
laravel-cms	2	200	Laravel	0.1029	30.43
laravel-cms	3	200	Laravel	0.0805	30.31
laravel-cms	4	200	Laravel	0.0253	30.59
laravel-cms	5	200	Laravel	0.0461	30.44

Fuente: `tests/results/20261013T203602Z/runs.jsonl`, transformado por `export_annex.py`.

Anexo E. Protocolos propuestos para la etapa comparativa y la evaluación con PYMEs

Comparación técnica. Antes de iniciar, fijar la versión y el hash de una distribución de OWASP ZAP, la lista de reglas de escaneo activo y pasivo, la política de autenticación, el presupuesto máximo de peticiones y el mapeo de alertas a las dos clases DAST y tres observaciones de cabeceras. Ejecutar ambos escáneres sobre las mismas URLs locales y sobre casos positivos y negativos construidos independientemente. Alternar el orden de herramientas por caso y repetir cinco veces para observar estabilidad. Conservar comandos, configuraciones, reportes crudos, tiempo de pared, memoria y solicitudes enviadas. Contar por separado clases que ZAP o el prototipo no prueben; no adjudicar un falso negativo a una función desactivada.

Presentar métricas por clase y un análisis de errores, sin convertirlas en una estimación de eficacia en PYMEs.

Estudio formativo con usuarios. Proponer al tutor la participación voluntaria de administradores o responsables web de PYMEs peruanas, con reclutamiento y consentimiento documentados antes de contactar a personas. Registrar rol, experiencia técnica y tipo de sitio sin publicar identificadores empresariales. En un entorno de demostración local se pediría instalar o iniciar el CLI, seleccionar módulos, reconocer que una alerta SQL es indicativa, distinguir una cabecera ausente de una explotación y elegir un siguiente paso apropiado. Medir éxito por tarea con criterios predefinidos, tiempo, errores de interpretación, necesidad de ayuda y valoración de claridad mediante preguntas abiertas. Una entrevista breve recogería barreras de adopción y costos de instalación, revisión y mantenimiento. El piloto no estimaría tasas poblacionales; cualquier prueba sobre sitios propios requeriría permiso explícito de la empresa, alcance acordado y separación de esos datos respecto del laboratorio.

Decisión de diseño pendiente. Si la comparación técnica es parte del objetivo y título aprobados por el tutor, debe ejecutarse antes de presentar un resultado comparativo. Si el objetivo se limita al artefacto y sus bancos controlados, el título propuesto en la portada describe la evidencia disponible y este anexo queda como protocolo futuro. La eventual validación con PYMEs constituye una etapa distinta en cualquiera de las dos alternativas.

Anexo F. Corridas completas del banco de mecanismos independientes

La tabla F.1 procede de `tests/results/20261013T203541Z-independent/runs.jsonl`; la verdad por ruta y el hash del servidor constan en su `manifest.json`. «Pared» incluye invocación del proceso CLI; «interno» no incluye arranque ni reporte. XSS y SQLI son indicios emitidos, no etiquetas de verdad. El JSON crudo conserva el detalle completo de cada hallazgo.

Tabla F.1. Treinta y cinco corridas del banco de mecanismos independientes

Ruta	Rep.	Indicios	Pared (s)	Interno (s)	Pico (MiB)
xss-html	1	XSS	0.5374	0.0155	30.24
xss-html	2	XSS	0.9655	0.0271	30.12
xss-html	3	XSS	0.8321	0.0622	30.17
xss-html	4	XSS	0.5389	0.0167	30.28
xss-html	5	XSS	0.4817	0.0225	30.15
xss-escaped	1	ninguno	0.7564	0.0174	30.20
xss-escaped	2	ninguno	0.6367	0.0387	30.16
xss-escaped	3	ninguno	0.5472	0.0252	30.21
xss-escaped	4	ninguno	0.4742	0.0556	30.23
xss-escaped	5	ninguno	0.4255	0.0095	30.26

Ruta	Rep.	Indicios	Pared (s)	Interno (s)	Pico (MiB)
xss-inert-text	1	ninguno	0.3627	0.0077	30.12
xss-inert-text	2	ninguno	0.3525	0.0067	30.22
xss-inert-text	3	ninguno	0.3422	0.0062	30.32
xss-inert-text	4	ninguno	0.3500	0.0085	30.15
xss-inert-text	5	ninguno	0.3915	0.0314	30.25
sqli-visible	1	SQLI	0.4778	0.0683	30.30
sqli-visible	2	SQLI	0.3904	0.0254	30.28
sqli-visible	3	SQLI	0.4152	0.0494	30.23
sqli-visible	4	SQLI	0.4476	0.0504	30.20
sqli-visible	5	SQLI	0.4279	0.0129	30.19
sqli-fake-error	1	SQLI	0.4107	0.0103	30.08
sqli-fake-error	2	SQLI	0.4185	0.0365	30.25
sqli-fake-error	3	SQLI	0.4253	0.0598	30.31
sqli-fake-error	4	SQLI	0.3936	0.0242	30.21
sqli-fake-error	5	SQLI	0.3834	0.0103	30.24
sqli-prepared	1	ninguno	0.4242	0.0117	30.24
sqli-prepared	2	ninguno	0.4399	0.0269	30.18
sqli-prepared	3	ninguno	0.4915	0.0349	30.20
sqli-prepared	4	ninguno	0.4452	0.0423	30.14
sqli-prepared	5	ninguno	0.4363	0.0459	30.36
sqli-hidden	1	ninguno	0.4501	0.0445	30.21
sqli-hidden	2	ninguno	0.4134	0.0333	30.26
sqli-hidden	3	ninguno	0.4099	0.0225	30.37
sqli-hidden	4	ninguno	0.4307	0.0416	30.15
sqli-hidden	5	ninguno	0.4563	0.0420	30.18

Fuente: `tests/results/20261013T203541Z-independent/runs.jsonl`, transformado por `export_independent_annex.py`.